Provenance Ledger

Concentricity

This document is generated from the author's master and fresh Lean checks. Each statement is typeset from the master. Every axiom line below is Lean's literal output.

Master revision. `63a334bf3ffab91efd5db9b9745bd3c6f6c9036cfb651a1c2ff969291e4cb380`

Lean-source revision. `f4c7618ecf3ef70d76b2f0161d7ed7668554822787eb4291f4b2adb4b5d4cbfb`

1. Theorem - Meromorphic continuation and functional equation; Riemann \cite{Riemann1859}

Author's statement

The Dirichlet series $\zeta(s) = \sum_{n\geq 1} n^{-s}$, convergent for $\operatorname{Re} s > 1$, continues to a meromorphic function on $\mathbb{C}$ with a single simple pole, at $s = 1$; equivalently, a holomorphic map to the Riemann sphere

$$\zeta : \mathbb{C}^* \longrightarrow \mathbb{C}^*, \qquad \mathbb{C}^* = \mathbb{C} \cup \{\infty\} = S^2, \qquad \zeta(1) = \infty.$$

The completed function $\xi(s) = \frac{1}{2}s(s-1)\pi^{-s/2}\Gamma(s/2)\zeta(s)$ is entire and satisfies

$$\xi(s) = \xi(1-s), \qquad \xi(\bar{s}) = \overline{\xi(s)}.$$

Consequently, if ρ is a nontrivial zero of ζ then so are $\bar{\rho}$, $1-\rho$, and $1-\bar{\rho}$. The trivial zeros of ζ are at $s = -2, -4, \ldots$; the nontrivial zeros lie in the strip $0 < \operatorname{Re} s < 1$. The Riemann Hypothesis asserts that every nontrivial zero has $\operatorname{Re} s = \frac{1}{2}$.

Lean declaration. `riemannZeta_meromorphicOn`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_meromorphicOn
'riemannZeta_meromorphicOn' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `riemannZeta_orderAt_one`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_orderAt_one
'riemannZeta_orderAt_one' depends on axioms: [propext, Classical.choice, Quot.sound]
```

---

1. Theorem - Meromorphic continuation and functional equation; Riemann \cite{Riemann1859} (continued)

Lean declaration. riemannZeta_conj
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_conj
'riemannZeta_conj' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. riemannZeta_one_sub_zero
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_one_sub_zero
'riemannZeta_one_sub_zero' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. nontrivialZero_re_mem_Ioo
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms nontrivialZero_re_mem_Ioo
'nontrivialZero_re_mem_Ioo' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

2. Theorem - Infinite Euler product; {\cite[Ch.~1]{Titchmarsh86}}

Author's statement
For $\operatorname{Re} s > 1$,

$$\zeta(s) = \prod_{p \text{ prime}} \left(1 - p^{-s}\right)^{-1} = \exp\Big(\sum_{p} \sum_{k \geq 1} \tfrac{1}{k}\, p^{-ks}\Big),$$

the product taken over the infinitely many primes; it converges absolutely and is zero-free on $\operatorname{Re} s > 1$.

2. Theorem - Infinite Euler product; {\cite[Ch.~1]{Titchmarsh86}} (continued)

Lean declaration. zetaEulerLog
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaEulerLog
'zetaEulerLog' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC2_summable
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC2_summable
'zetaC2_summable' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC2_zero_free
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC2_zero_free
'zetaC2_zero_free' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC2_euler
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC2_euler
'zetaC2_euler' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

3. Theorem - Infinite Hadamard product; {\cite[Ch.~2]{Titchmarsh86}}

Author's statement

ξ is entire of order 1 with infinitely many zeros, which are exactly the nontrivial zeros $\{\rho\}$ of ζ, and admits the Hadamard factorization

$$\xi(s) = \xi(0) \prod_{\rho} \Big(1 - \tfrac{s}{\rho}\Big) e^{s/\rho},$$

the product taken over the infinitely many nontrivial zeros.

Lean declaration. `xi_entire`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms xi_entire
'xi_entire' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `xi_zeros_eq_nontrivialZeros`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms xi_zeros_eq_nontrivialZeros
'xi_zeros_eq_nontrivialZeros' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `zeta_hadamard`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeta_hadamard
'zeta_hadamard' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `riemannZeta_nontrivialZeros_infinite`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

3. Theorem - Infinite Hadamard product; {\cite[Ch.~2]{Titchmarsh86}} (continued)

```
#print axioms riemannZeta_nontrivialZeros_infinite
'riemannZeta_nontrivialZeros_infinite' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

4. Corollary - Infinitude of the nontrivial zeros

Author's statement
ζ has infinitely many nontrivial zeros.

Lean declaration. `riemannZeta_nontrivialZeros_infinite`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_nontrivialZeros_infinite
'riemannZeta_nontrivialZeros_infinite' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

5. Theorem - Hardy \cite{Hardy14}

Author's statement
Infinitely many nontrivial zeros of ζ lie on the line $\operatorname{Re} s = \frac{1}{2}$.

Lean declaration. `zeta_criticalLine_zeros_infinite`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeta_criticalLine_zeros_infinite
'zeta_criticalLine_zeros_infinite' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

6. Definition - Compactified classical zeta

Author's statement

$\zeta_{\mathbb{C}^*} : \mathbb{C}^* \to \mathbb{C}^*$ is the meromorphic continuation of ζ regarded as a map to the Riemann sphere: $\zeta_{\mathbb{C}^*}(s) = \zeta(s)$ for $s \in \mathbb{C} \setminus \{1\}$; $\zeta_{\mathbb{C}^*}(1) = \infty$ (the simple pole, Theorem 1.1); and $\zeta_{\mathbb{C}^*}(\infty) = 1$, since $\zeta(s) \to 1$ as $\mathrm{Re}(s) \to +\infty$ (the p-test: only the $n = 1$ term of $\sum_n n^{-s}$ survives). It is holomorphic into $\mathbb{C}^*$ on $\mathbb{C}^* \setminus \{1\}$, its sole pole being $s = 1$. The functional equation, the conjugate symmetry $\zeta_{\mathbb{C}^*}(\bar{s}) = \overline{\zeta_{\mathbb{C}^*}(s)}$, and Hardy's infinitude (Theorem 1.5) hold verbatim for $\zeta_{\mathbb{C}^*}$, as they concern values on $\mathbb{C} \setminus \{1\}$ where $\zeta_{\mathbb{C}^*} = \zeta$.

Lean declaration. zetaC
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC
'zetaC' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC_coe
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC_coe
'zetaC_coe' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC_one
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC_one
'zetaC_one' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaC_infty
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC_infty
'zetaC_infty' depends on axioms: [propext, Classical.choice, Quot.sound]
```

6. Definition - Compactified classical zeta (continued)

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

7. Lemma - Compactification preserves the zero set

Author's statement
$Z(\zeta_{\mathbb{C}^*}) = Z(\zeta)$: the zeros of $\zeta_{\mathbb{C}^*}$ in $\mathbb{C}^*$ are exactly the zeros of ζ in $\mathbb{C}$; in particular the nontrivial zeros coincide.

Lean declaration. `zetaC_coe_eq_zero_iff`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC_coe_eq_zero_iff
'zetaC_coe_eq_zero_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `zetaC_zero_iff`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaC_zero_iff
'zetaC_zero_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

8. Definition

Author's statement
The octonions are: (i) a *division algebra*: every nonzero element has a multiplicative inverse; (ii) a *normed algebra*: $|ab| = |a||b|$; (iii) *non-associative*: $(ab)c \neq a(bc)$ in general; (iv) *alternative*: $(aa)b = a(ab)$ and $a(bb) = (ab)b$.

Lean declaration. `Octonion`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

8. Definition (continued)

Lean command and literal axiom print.

```
#print axioms Octonion
'Octonion' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.normSq_mul
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.normSq_mul
'Octonion.normSq_mul' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.alt_left
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.alt_left
'Octonion.alt_left' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.alt_right
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.alt_right
'Octonion.alt_right' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

9. Theorem - Artin \cite{Schafer66}

Author's statement
In an alternative algebra, the subalgebra generated by any two elements is associative.

9. Theorem - Artin \cite{Schafer66} (continued)

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

10. Corollary

Author's statement
Powers s^n are well-defined for any $s \in \mathbb{O}$.

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

11. Definition - Complex slices and slice Riemann spheres

Author's statement
For any unit imaginary octonion $v \in S^6$, the *complex slice* is $\mathbb{C}_v = \{a + bv : a, b \in \mathbb{R}\} \subset \mathbb{O}$. Since $v^2 = -1$, this is a subalgebra isomorphic to $\mathbb{C}$ via the $\mathbb{R}$-algebra isomorphism $\varphi_v : \mathbb{C} \to \mathbb{C}_v$, $i \mapsto v$. All slices share the real axis: $\mathbb{C}_v \cap \mathbb{C}_w = \mathbb{R}$ for $v \neq \pm w$. Its one-point compactification is the *slice Riemann sphere* $\mathbb{C}_v^* := \mathbb{C}_v \cup \{\infty\} = S_v^2$, and φ_v extends to an isomorphism of Riemann spheres $\varphi_v : \mathbb{C}^* \xrightarrow{\sim} \mathbb{C}_v^*$ with $\varphi_v(\infty) = \infty$ (GPV [17], Prop. 4.1/Thm. 4.2). All slice spheres share the single point at infinity, as well as the real axis.

Lean declaration. `Octonion.unitImaginarySphere`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.unitImaginarySphere
'Octonion.unitImaginarySphere' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `Octonion.sliceEmbed`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.sliceEmbed
'Octonion.sliceEmbed' depends on axioms: [propext, Classical.choice, Quot.sound]
```

11. Definition - Complex slices and slice Riemann spheres (continued)

Lean declaration. `Octonion.sliceEmbed_mul`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.sliceEmbed_mul
'Octonion.sliceEmbed_mul' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `Octonion.sliceSphere`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.sliceSphere
'Octonion.sliceSphere' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

12. Definition - Slice regularity; {\cite[Def.~5.1.1]{CSS12}}

Author's statement
Let $\Omega \subseteq \mathbb{O}$ be open. A function $f : \Omega \to \mathbb{O}$ is *slice regular* if for each $v \in S^6$, $\varphi_v^{-1} \circ f \circ \varphi_v$ satisfies the Cauchy–Riemann equations on $\varphi_v^{-1}(\Omega \cap \mathbb{C}_v) \subseteq \mathbb{C}$. A domain Ω is *axially symmetric* if $\sigma + \gamma v \in \Omega$ implies $\sigma + \gamma w \in \Omega$ for all $w \in S^6$.

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

13. Theorem - Representation Formula; {\cite[Thm.~5.1.7]{CSS12}}

Author's statement
Let f be slice regular on an axially symmetric slice domain Ω. Fix $v_0 \in S^6$ and write $f(\sigma + \gamma v_0) = \alpha(\sigma, \gamma) + v_0 \cdot \beta(\sigma, \gamma)$ with $\alpha, \beta : \mathbb{R}^2 \to \mathbb{R}$. Then for every $v \in S^6$, $f(\sigma + \gamma v) = \alpha(\sigma, \gamma) + v \cdot \beta(\sigma, \gamma)$, where $\alpha = \frac{1}{2}[f(\sigma + \gamma v_0) + f(\sigma - \gamma v_0)]$ and $\beta = \frac{1}{2} v_0^{-1}[f(\sigma - \gamma v_0) - f(\sigma + \gamma v_0)]$.

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author bind-

13. Theorem - Representation Formula; {\cite[Thm.~5.1.7]{CSS12}} (continued)

ing registry does not match the current master

14. Theorem - Identity Theorem; {\cite[Cor.~5.1.9]{CSS12}}

Author's statement

Let f be slice regular on an axially symmetric slice domain Ω. If f vanishes on a subset of a single slice $\Omega \cap \mathbb{C}_{v_0}$ with an accumulation point in $\Omega \cap \mathbb{C}_{v_0}$, then $f \equiv 0$ on all of Ω.

Lean declaration. `stem_identity`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms stem_identity
'stem_identity' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

15. Theorem - Extension Theorem; {\cite[Thm.~5.1.5]{CSS12}}

Author's statement

Let $U \subseteq \mathbb{C}$ be a domain symmetric under conjugation and $g : U \to \mathbb{C}$ holomorphic. There is a unique slice regular $\mathrm{ext}(g) : \Omega_U \to \mathbb{O}$ on $\Omega_U = \{\sigma + \gamma v : \sigma + i\gamma \in U,\ v \in S^6\}$ with $\mathrm{ext}(g)|_{\mathbb{C}_{v_0}} = \varphi_{v_0} \circ g \circ \varphi_{v_0}^{-1}$ for every v_0.

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

16. Definition - Slice-preserving functions; \protect{\cite[Def.~2.7, Rem.~2.8]{AdF}; \cite{VS,SeriesExp,Wang}}

Author's statement

A slice-regular function f on an axially symmetric domain Ω is *slice preserving* if it maps every slice into itself: $f(\Omega \cap \mathbb{C}_v) \subseteq \mathbb{C}_v$ for all $v \in S^6$. For octonions ($\#S^6 > 2$) the following are equivalent characterisations:

(i) f is *intrinsic*: $f(q^c) = f(q)^c$ for all $q \in \Omega$, where q^c is octonionic conjugation (on a slice, $a + bv \mapsto a - bv$);

(ii) its representation-formula components are $\mathbb{R}$-valued: writing $f(\sigma + \gamma v) = \alpha(\sigma, \gamma) +$

16. Definition - Slice-preserving functions; \protect{\cite[Def.˜2.7, Rem.˜2.8]{AdF}; \cite{VS,SeriesExp,Wang}} (continued)

$v\,\beta(\sigma,\gamma)$ (Theorem 3.2), one has $\alpha,\beta:\mathbb{R}^2\to\mathbb{R}$, equivalently the inducing stem function $F=F_1+\iota F_2$ has $\mathbb{R}$-valued components F_1,F_2;

(iii) f preserves two distinct slices; equivalently $f(\Omega\cap\mathbb{R})\subseteq\mathbb{R}$ when Ω is a slice domain.

Slice-preserving functions are the more restrictive subclass of slice-regular functions on which classical complex methods apply slicewise. They are precisely the sections of $\mathcal{R}$ (Definition 5.1), and on them the regular product is the ordinary pointwise product, so the product is commutative (Theorem 5.2; Ghiloni–Perotti– Stoppato [7, Rem. 1.8]).

Lean declaration. `IsIntrinsic`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms IsIntrinsic
'IsIntrinsic' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `StemRing.isIntrinsic`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms StemRing.isIntrinsic
'StemRing.isIntrinsic' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `StemRing.real_on_real`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms StemRing.real_on_real
'StemRing.real_on_real' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

17. Definition - Octonionic zeta

Author's statement

Here N denotes the *north pole* of S^8: the single point at infinity ∞ adjoined in the one-point compactification $\mathbb{O}^* = \mathbb{O} \cup \{\infty\} \cong S^8$ (inverse stereographic projection; GPV [17]). It is one point with two names, so $\infty = N$; it is the only point at infinity, shared by $\mathbb{O}^*$ and by every slice Riemann sphere $\mathbb{C}_v^* = S_v^2$.

The *octonionic zeta function* $\zeta_{\mathbb{O}^*} : \mathbb{O}^* \to \mathbb{O}^*$ is defined slicewise from the compactified classical zeta (Definition 1.6) through the slice identifications $\varphi_v : \mathbb{C}^* \xrightarrow{\sim} \mathbb{C}_v^* = S_v^2$ (the $\mathbb{R}$-algebra isomorphism $\mathbb{C} \to \mathbb{C}_v$ of Definition 2.4, extended to the Riemann spheres by $\varphi_v(\infty) = N$; GPV [17], Prop. 4.1/Thm. 4.2):

(i) for $s \in \mathbb{O} \setminus \mathbb{R}$, with $w = \mathrm{Im}(s)$, $v = w/|w| \in S^6$, $\gamma = |w| > 0$, $\sigma = \mathrm{Re}(s)$, so $s = \sigma + \gamma v$: $\zeta_{\mathbb{O}^*}(s) := \varphi_v(\zeta_{\mathbb{C}^*}(\sigma + i\gamma))$;

(ii) for $s \in \mathbb{R} \setminus \{1\}$: $\zeta_{\mathbb{O}^*}(s) := \zeta_{\mathbb{C}^*}(s) = \zeta(s) \in \mathbb{R}$;

(iii) for $s = 1$ (the simple pole, Definition 1.6): $\zeta_{\mathbb{O}^*}(1) := \infty = N$;

(iv) for $s = \infty = N$: $\zeta_{\mathbb{O}^*}(\infty) := \zeta_{\mathbb{C}^*}(\infty) = 1$.

Lean declaration. `zetaO`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO
'zetaO' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `zetaO_infty`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_infty
'zetaO_infty' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `zetaO_one`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_one
'zetaO_one' depends on axioms: [propext, Classical.choice, Quot.sound]
```

17. Definition - Octonionic zeta (continued)

Lean declaration. zetaO_sliceEmbed
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_sliceEmbed
'zetaO_sliceEmbed' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaO_sliceEmbed_of_im_neg
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_sliceEmbed_of_im_neg
'zetaO_sliceEmbed_of_im_neg' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. zetaO_ofReal
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_ofReal
'zetaO_ofReal' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

18. Proposition - Well-definedness

Author's statement
$\zeta_{\mathbb{O}^*} : \mathbb{O}^* \to \mathbb{O}^*$ is well-defined.

Lean declaration. zetaO_sliceEmbed'
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

18. Proposition - Well-definedness (continued)

Lean command and literal axiom print.

```
#print axioms zeta0_sliceEmbed'
'zeta0_sliceEmbed'' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. riemannZeta_conj
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannZeta_conj
'riemannZeta_conj' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

19. Definition - The ring of slice-preserving functions under the regular product

Author's statement
$\mathcal{R} := \mathcal{O}_{\mathbb{O}^*}$ is the set of all slice-regular functions $f : \mathbb{O}^* \to \mathbb{O}^*$ that are slice preserving (Definition 3.5), $f(\Omega_v) \subseteq \mathbb{C}_v^*$ for all $v \in S^6$, equipped with pointwise addition and the regular $(*)$ product.

Lean declaration. StemRing
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms StemRing
'StemRing' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

20. Theorem - Regular product on slice-preserving functions; {\cite[Rem.~2.11]{Wang}}

Author's statement
For slice-preserving f, g (with $f(\Omega_v) \subseteq \mathbb{C}_v$), the regular product restricts on each slice to the pointwise product, $(f * g)|_{\Omega_v}(w) = f(w) \cdot g(w) \in \mathbb{C}_v$, and $f * g = g * f$, $(f * g) * h = f * (g * h)$

20. Theorem - Regular product on slice-preserving functions; {\cite[Rem.~2.11]{Wang}} (continued)

(via Artin, Theorem 2.2).

No exact declaration/object check is available for this clause.

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

21. Proposition

Author's statement
$(\mathcal{R}, +, *)$ is a commutative ring with identity $1 \in \mathcal{R}$.

Lean declaration. `StemRing`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms StemRing
'StemRing' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

22. Proposition - $\mathcal R$ is an integral domain

Author's statement
If $f, g \in \mathcal{R}$ and $f * g = 0$, then $f = 0$ or $g = 0$.

Lean declaration. `StemRing.eq_zero_or_eq_zero_of_mul_eq_zero`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms StemRing.eq_zero_or_eq_zero_of_mul_eq_zero
'StemRing.eq_zero_or_eq_zero_of_mul_eq_zero' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

23. Theorem - The slice exponential; \protect{\cite[\S3]{GPVwind}; \cite[Prop.˜5.1]{VS}; \cite[Rem.˜2.23]{AdF}}

Author's statement

The exponential $\exp : \mathbb{O} \to \mathbb{O} \setminus \{0\}$, $\exp(q) = \sum_{k\geq 0} q^k/k!$ (powers well defined by Corollary 2.3), is a slice-regular, slice-preserving entire function (Definition 3.5), given on each slice by

$$\exp(x + Iy) = e^x(\cos y + I \sin y), \qquad x, y \in \mathbb{R},\ I \in S^6.$$

Consequently $|\exp q| = e^{\operatorname{Re} q}$ depends only on the real part, and exp is nowhere zero.

Lean declaration. `Octonion.exp`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.exp
'Octonion.exp' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `Octonion.exp_sliceEmbed`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.exp_sliceEmbed
'Octonion.exp_sliceEmbed' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `Octonion.norm_exp`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.norm_exp
'Octonion.norm_exp' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `Octonion.exp_ne_zero`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

23. Theorem - The slice exponential; \protect{\cite[\S3]{GPVwind}; \cite[Prop.~5.1]{VS}; \cite[Rem.~2.23]{AdF}} (continued)

```
#print axioms Octonion.exp_ne_zero
'Octonion.exp_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

24. Theorem - The logarithm manifold $E^{+}_{\OO}$; \protect{\cite[Prop.~5.1, Rem.~5.2, Def.~5.3, Prop.~5.4, Def.~5.5]{VS}; \cite[\S3]{GPVwind}}

Author's statement

Let $T : \mathbb{O} \to \mathbb{O} \times \operatorname{Im}\mathbb{O}$, $T(x+Iy) = (\sinh x \cos y + I \sinh x \sin y,\ Iy)$, and $\mathbb{O}^+ = \{q : \operatorname{Re} q > 0\}$. The *logarithm manifold* is $E^+_{\mathbb{O}} := T(\mathbb{O}^+)$, the semi-helicoidal hypercomplex 8-manifold; equivalently it is the image of the exponential graph, parametrised by the $E^+_{\mathbb{O}}$*-exponential*

$$E : \mathbb{O} \longrightarrow E^+_{\mathbb{O}} \subset \mathbb{O} \times \operatorname{Im}\mathbb{O}, \qquad E(x+Iy) = (\exp(x+Iy),\ Iy) = (e^x \cos y + Ie^x \sin y,\ Iy),$$

an immersion and a diffeomorphism, with inverse the $E^+_{\mathbb{O}}$*-logarithm* $L : E^+_{\mathbb{O}} \to \mathbb{O}$, $L(q,p) = \log|q| + p$ (here $p = \operatorname{Arg}(q)$). Writing π for the projection to the first factor, $\pi \circ E = \exp$ (Theorem 6.1). The manifold $E^+_{\mathbb{O}}$ is an adapted blow-up of $\mathbb{O}$ along the real axis. Its projection $\pi : E^+_{\mathbb{O}} \to \mathbb{O} \setminus \{0\}$ has the degenerate real fibres responsible for the failure of covering and openness. The blow-up unwraps the multivalued logarithm into the single-valued L, and a branch of the hypercomplex logarithm is $\log_{\mathbb{O}} = L \circ (\pi|_\Omega)^{-1}$ on any path-connected $\Omega \subset E^+_{\mathbb{O}}$ on which $\pi|_\Omega$ is injective.

Lean declaration. Octonion.Eexp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.Eexp
'Octonion.Eexp' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.logManifold
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.logManifold
'Octonion.logManifold' depends on axioms: [propext, Classical.choice, Quot.sound]
```

24. Theorem - The logarithm manifold $E^{+}_{\OO}$; \protect{\cite[Prop.~5.1, Rem.~5.2, Def.~5.3, Prop.~5.4, Def.~5.5]{VS}; \cite[\S3]{GPVwind}} (continued)

Lean declaration. Octonion.Llog
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.Llog
'Octonion.Llog' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.Llog_Eexp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.Llog_Eexp
'Octonion.Llog_Eexp' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.Eexp_Llog
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.Eexp_Llog
'Octonion.Eexp_Llog' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. Octonion.exp_Llog
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.exp_Llog
'Octonion.exp_Llog' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

25. Lemma - The degenerate set of the exponential; its fibre over a real value

Author's statement

Sourced. With π the first-factor projection, $\pi \circ E = \exp$ ([17, Rem. 5.2(a)], the commuting triangle). Its spherical *degenerate set* makes the projection an adapted blow-up with failures of covering and openness ([17, Rem. 5.2(b)]). The direction $I(q)$ becomes singular at every point of $\mathbb{R}$ ([21, Rem. 2.1]).

Proved from the slice form. For $r > 0$,

$$\exp^{-1}(-r) = \left\{ \log r + I(2k{+}1)\pi : \ I \in S^6, \ k \in \mathbb{Z} \right\} :$$

by Theorem 6.1, $\exp(x + Iy) = e^x \cos y + Ie^x \sin y = -r$ forces $e^x \sin y = 0$, hence $y \in \pi\mathbb{Z}$; negativity of the value forces $y = (2k{+}1)\pi$ and $x = \log r$; the direction I is unconstrained. The fibre is thus indexed by the single real *level* $\log r = \log|-r|$ and, within it, by the odd winding indices $2k{+}1$ carried as band data (the winding lift, Theorem 6.5; [21, Cor. 5.21]). The fibre formula itself appears, stated without proof as Preface motivation, in [17] (p. 972); the slice-form derivation above is the load-bearing statement.

Lean declaration. `Octonion.exp_fibre_neg_real`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.exp_fibre_neg_real
'Octonion.exp_fibre_neg_real' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `Octonion.exp_eq_neg_ofReal_iff`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.exp_eq_neg_ofReal_iff
'Octonion.exp_eq_neg_ofReal_iff' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `Octonion.logManifold_fibre_neg_real`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.logManifold_fibre_neg_real
'Octonion.logManifold_fibre_neg_real' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

25. Lemma - The degenerate set of the exponential; its fibre over a real value (continued)

Lean declaration. `Octonion.degenerate_level_readout`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.degenerate_level_readout
'Octonion.degenerate_level_readout' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

26. Definition - Slice restriction and the I-independent lift; \protect{\cite[Rem.~2.11]{Wang}; \cite[\S3.3,~\S3.7]{BisiWinkelmann}; \cite[Def.~2.1, Prop.~2.2]{SeriesExp}}

Author's statement
A slice-preserving section A carries each slice Riemann sphere into itself,

$$A(\mathbb{C}_v^*) \subseteq \mathbb{C}_v^* = S_v^2 \qquad (v \in S^6),$$

so its restriction to a slice is a holomorphic self-map of that slice sphere,

$$A_v := \varphi_v^{-1} \circ A \circ \varphi_v : \ \mathbb{C}^* \longrightarrow \mathbb{C}^*,$$

through the identification $\varphi_v : \mathbb{C}^* \xrightarrow{\sim} \mathbb{C}_v^*$ of Definition 2.4. The *I-independent lift* is the fact that this holomorphic stem is the *same* for every v: there is one holomorphic $A_\circ : \mathbb{C}^* \to \mathbb{C}^*$ with

$$A|_{\mathbb{C}_v^*} = \varphi_v \circ A_\circ \circ \varphi_v^{-1} \qquad \text{for all } v \in S^6$$

(Wang [16, Rem. 2.11]; Bisi–Winkelmann [4, §3.7]). Equivalently, A is *intrinsic*, $A(q^c) = A(q)^c$, and is induced ($A = \mathcal{I}(F)$) by a single stem function $F = F_1 + \iota F_2 : D \to \mathbb{O}_{\mathbb{C}}$ on the symmetric $D \subset \mathbb{C}$ whose components F_1, F_2 are $\mathbb{R}$-valued (the stem-function formalism, Ghiloni–Perotti [7, Def. 2.1, Prop. 2.2]):

$$A(x + Iy) = F_1(x + iy) + I\, F_2(x + iy), \qquad F_1, F_2 \ \mathbb{R}\text{-valued}.$$

It is this single I-independent slice stem that makes a slice-preserving section G_2-equivariant, hence blind to the G_2-orbit direction (Definition 9.2).

Lean declaration. `IsIntrinsic`
Author's object.
Object relation. Object relation not checked

26. Definition - Slice restriction and the I-independent lift; \protect{\cite[Rem.~2.11]{Wang}; \cite[\S3.3,~\S3.7]{BisiWinkelmann}; \cite[Def.~2.1, Prop.~2.2]{SeriesExp}} (continued)

Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms IsIntrinsic
'IsIntrinsic' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.realize
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize
'ASection.realize' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.realize_mem_sliceSphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize_mem_sliceSphere
'ASection.realize_mem_sliceSphere' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.realize_equivariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize_equivariant
'ASection.realize_equivariant' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

27. Theorem - The winding lift; \protect{\cite[Def.˜3.4, Def.˜4.1, Prop.˜4.2, Def.˜5.11]{GPVwind}}

Author's statement

Let $\gamma : [a,b] \to \mathbb{O} \setminus \{0\}$ be a path (on each slice $\mathbb{C}_v$ this is the classical complex continuation). A *continuation of the hypercomplex logarithm along* γ is a path $\tilde{\gamma} : [a,b] \to \mathbb{O}$ with $\exp \circ \tilde{\gamma} = \gamma$, and a *lift of* γ to the logarithm manifold $E_{\mathbb{O}}^{+}$ (Theorem 6.2) is a path $\Gamma : [a,b] \to E_{\mathbb{O}}^{+}$ with $\mathrm{pr}_1 \circ \Gamma = \gamma$:

$$\begin{array}{ccc} & & \mathbb{O} \\ & \overset{\tilde{\gamma}}{\nearrow} & \uparrow{\scriptstyle \exp} \\ [a,b] & \xrightarrow[\gamma]{} & \mathbb{O} \setminus \{0\} \end{array} \qquad \begin{array}{ccc} & & E_{\mathbb{O}}^{+} \\ & \overset{\Gamma}{\nearrow} & \downarrow{\scriptstyle \mathrm{pr}_1} \\ [a,b] & \xrightarrow[\gamma]{} & \mathbb{O} \setminus \{0\} \end{array}$$

The two data are *equivalent*: a lift Γ exists if and only if a continuation $\tilde{\gamma}$ exists, and they correspond by

$$\tilde{\gamma} = L \circ \Gamma, \qquad \Gamma = (\exp \circ \tilde{\gamma},\ \mathrm{Im}\, \tilde{\gamma}),$$

where L is the $E_{\mathbb{O}}^{+}$-logarithm of Theorem 6.2 ([21, Prop. 4.2]). For a *loop* $\gamma : S^1 \to \mathbb{O} \setminus \{0\}$ the lift is taken on the universal cover $\exp : i\mathbb{R} \to S^1$, as a continuous $\Gamma : i\mathbb{R} \to E_{\mathbb{O}}^{+}$ with $\mathrm{pr}_1 \circ \Gamma = \gamma \circ \exp$ ([21, Def. 5.11]):

$$\begin{array}{ccc} i\mathbb{R} & \xrightarrow{\Gamma} & E_{\mathbb{O}}^{+} \\ {\scriptstyle \exp}\downarrow & & \downarrow{\scriptstyle \mathrm{pr}_1} \\ S^1 & \xrightarrow[\gamma]{} & \mathbb{O} \setminus \{0\} \end{array}$$

Lean declaration. `exists_log_continuation`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms exists_log_continuation
'exists_log_continuation' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `winding_lift_unique`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms winding_lift_unique
'winding_lift_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `Octonion.lift_iff_continuation`
Author's object.

27. Theorem - The winding lift; \protect{\cite[Def.˜3.4, Def.˜4.1, Prop.˜4.2, Def.˜5.11]{GPVwind}} (continued)

Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion.lift_iff_continuation
'Octonion.lift_iff_continuation' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

28. Proposition - Tameness, existence, and the winding number; \protect{\cite[Def.˜4.20, Cor.˜5.21, Cor.˜5.22]{GPVwind}}

Author's statement
The lift of Theorem 6.5 is governed by the companion structure of γ.

(i) *Uniqueness (tameness).* γ is *tame* when it admits a *unique* companion imaginary-unit field; its lift is then unique ([21, Def. 4.20]).

(ii) *Existence and closure.* A lift of a loop γ exists if and only if the signature satisfies $\sigma(\gamma|_{[\xi_l,\xi_{l+1}]}) \in \{0,-1\}$ on each obstruction interval; and *if it exists, the lift is itself a loop* ([21, Cor. 5.13]; pinned against the arXiv text; the earlier citation to Cor. 5.22 is superseded).

(iii) *Winding number.* For an untwisted tame loop with even circular signature σ^c, the winding number is $\omega = |\sigma^c|/2$ ([21, Cor. 5.21]).

Lean declaration. `winding_lift_unique`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms winding_lift_unique
'winding_lift_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `winding_loop_defect`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

28. Proposition - Tameness, existence, and the winding number; \protect{\cite[Def.~4.20, Cor.~5.21, Cor.~5.22]{GPVwind}} (continued)

```
#print axioms winding_loop_defect
'winding_loop_defect' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

29. Theorem - Slice preservation

Author's statement

$\zeta_{\mathbb{O}^*}$ is slice preserving (Definition 3.5): $\zeta_{\mathbb{O}^*}(\mathbb{C}_v^*) \subseteq \mathbb{C}_v^*$ for every $v \in S^6$.

Lean declaration. zetaO_mem_sliceSphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_mem_sliceSphere
'zetaO_mem_sliceSphere' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

30. Theorem - ζO is a section of $\mathcal R$

Author's statement

$\zeta_{\mathbb{O}^*} \in \mathcal{R}$.

Lean declaration. zetaSection
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaSection
'zetaSection' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

31. Definition - The automorphism group $\Gtwo$

Author's statement
$G_2 = \mathrm{Aut}(\mathbb{O})$ is the 14-dimensional compact, connected, simply connected Lie group of $\mathbb{R}$-algebra automorphisms of $\mathbb{O}$. Each $g \in G_2$ fixes 1, hence the real axis pointwise, and acts on $\mathrm{Im}\,\mathbb{O} \cong \mathbb{R}^7$ as an element of $SO(7)$; thus $g \cdot (\sigma + \gamma v) = \sigma + \gamma\, g(v)$.

Lean declaration. G2
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms G2
'G2' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. G2.smul_one
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms G2.smul_one
'G2.smul_one' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. G2.smul_ofReal
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms G2.smul_ofReal
'G2.smul_ofReal' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

32. Theorem - $\Gtwo$-equivariance

Author's statement
$\zeta_{\mathbb{O}^*}(g \cdot s) = g \cdot \zeta_{\mathbb{O}^*}(s)$ for all $g \in G_2$, $s \in \mathbb{O}$.

32. Theorem - $\Gtwo$-equivariance (continued)

Lean declaration. zeta0_equivariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeta0_equivariant
'zeta0_equivariant' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

33. Theorem - \protect{\cite{Baez02}}

Author's statement
G_2 acts transitively on S^6 with stabilizer SU(3): $\mathrm{SU}(3) \hookrightarrow G_2 \twoheadrightarrow S^6$.

Lean declaration. G2.exists_smul_eq_of_mem_unitImaginarySphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms G2.exists_smul_eq_of_mem_unitImaginarySphere
'G2.exists_smul_eq_of_mem_unitImaginarySphere' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

34. Theorem - Zero Equivalence Theorem

Author's statement
Let $\rho = \sigma + i\gamma \in \mathbb{C}^*$. For every $v \in S^6$,

$$\zeta_{\mathbb{O}^*}(\sigma + \gamma v) = 0 \quad \Longleftrightarrow \quad \zeta_{\mathbb{C}^*}(\rho) = 0.$$

Equivalently: a non-real $s \in \mathbb{O}^*$ is a zero of $\zeta_{\mathbb{O}^*}$ if and only if its slice coordinate $\varphi_v^{-1}(s)$ is a zero of $\zeta_{\mathbb{C}^*}$; by Lemma 1.7 these are exactly the nontrivial zeros of the classical ζ.

Lean declaration. sliceEmbed_eq_zero_iff
Author's object.

34. Theorem - Zero Equivalence Theorem (continued)

Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms sliceEmbed_eq_zero_iff
'sliceEmbed_eq_zero_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zetaO_zero_iff
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaO_zero_iff
'zetaO_zero_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

35. Theorem - Nontrivial zeros are infinitely many disjoint 6-spheres

Author's statement
For a nontrivial zero $\rho = \sigma + i\gamma$ of $\zeta_{\mathbb{C}^*}$ (so $\gamma > 0$) set

$$S_\rho = \{\sigma + \gamma v : v \in S^6\} = \sigma + \gamma\, S^6 \subset \mathbb{O}.$$

Then:

(i) S_ρ *is* a 6-sphere, the round sphere of radius γ centred at the real point σ, equivalently the G_2-orbit of any one of its points $\sigma + \gamma v_0$ (Theorem 7.4; G_2 acts transitively on S^6 with stabiliser SU(3), Theorem 7.5, Baez [3]).

(ii) every point of S_ρ is a zero of $\zeta_{\mathbb{O}^*}$, and conjugate zeros give the same sphere, $S_\rho = S_{\bar\rho}$;

(iii) the nontrivial-zero set is the disjoint union $Z^{\mathrm{nt}}_{\mathbb{O}^*} = \bigsqcup_{[\rho]} S_\rho$ over conjugate pairs $[\rho] = \{\rho, \bar\rho\}$, with distinct pairs giving disjoint spheres;

(iv) there are infinitely many such spheres (Corollary 1.4; also Hardy [11], Theorem 1.5).

Lean declaration. zeroSphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

35. Theorem - Nontrivial zeros are infinitely many disjoint 6-spheres (continued)

Lean command and literal axiom print.

```
#print axioms zeroSphere
'zeroSphere' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zeroSphere_eq_orbit
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeroSphere_eq_orbit
'zeroSphere_eq_orbit' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zeroSphere_zeros
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeroSphere_zeros
'zeroSphere_zeros' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zeroSphere_disjoint
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeroSphere_disjoint
'zeroSphere_disjoint' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. zeroSpheres_infinite
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeroSpheres_infinite
'zeroSpheres_infinite' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

36. Theorem - The equivalence: concentricity $\Leftrightarrow$ RH

Author's statement
The following are equivalent:

(a) the Riemann Hypothesis;

(b) the nontrivial-zero 6-spheres $\{S_\rho\}$ of $\zeta_{\mathbb{O}^*}$ (Theorem 8.2) are *concentric*: they share a single common centre, necessarily a point of the real axis.

When they hold, the common centre is $\frac{1}{2}$.

Lean declaration. `riemannHypothesis_iff_concentric`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms riemannHypothesis_iff_concentric
'riemannHypothesis_iff_concentric' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

37. Definition - The compactified octonions $\OO^{*}=S^8$

Author's statement
Let $\mathbb{O}^* = \mathbb{O} \cup \{\infty\} = S^8$ be the Alexandroff one-point compactification of $\mathbb{O} = \mathbb{R}^8$. Concretely (GPV [17], Prop. 4.1 and Thm. 4.2, for $m = \dim_{\mathbb{R}} K \in \{4, 8\}$): the inverse stereographic projections from the north and south poles,

$$f(x+Iy) = \left(\tfrac{2(x+Iy)}{1+x^2+y^2},\ \tfrac{-1+x^2+y^2}{1+x^2+y^2}\right), \qquad h(x+Iy) = \left(\tfrac{2(x-Iy)}{1+x^2+y^2},\ \tfrac{1-x^2-y^2}{1+x^2+y^2}\right),$$

give a conformal atlas $\{(K, f), (K, h)\}$ endowing S^m with the structure of a slice (quaternionic/octonionic) Riemann manifold, with transition map $h^{-1} \circ f(q) = \bar{q}/q^2 = 1/q$, itself slice regular and slice preserving. The adjoined point is the north pole N; this is the point at infinity. In Lean the underlying compactification is `Mathlib.Topology.Compactification.OnePoint`, $\mathbb{O}^* =$ `OnePoint` $\mathbb{O}$ with $\infty =$ `OnePoint.infty`, and the homeomorphism $\mathbb{O}^* \cong S^8$ is `onePointEquivSphereOfFinrankEq` applied to $\dim_{\mathbb{R}} \mathbb{O} = 8$ (`Mathlib.Topology.Compactification.OnePoint.Sphere`). The compactification is what makes the value of a section at its real pole a well-defined point of $\mathbb{O}^*$, the point at infinity $\infty = N$. The two views $\mathcal{H}_1$ and $\mathcal{S}_2$ are built on these compactified values (Definition 9.3); the distinct projective base $\mathcal{B}$ and direction-indexed sphere world used in the construction below are introduced in Definition 9.4.

37. Definition - The compactified octonions $\OO^{*}=S^8$ (continued)

Lean declaration. `Octonion`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms Octonion
'Octonion' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `onePointEquivSphereOfFinrankEq`
Author's object.
Object relation. Object relation not checked
Exact state. External library dependency; approved kernel-reported axiom surface; no authored-object anchor required

Lean command and literal axiom print.

```
#print axioms onePointEquivSphereOfFinrankEq
'onePointEquivSphereOfFinrankEq' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

38. Definition - The section's slice data and equivariance

Author's statement
Let $A \in \mathcal{R}$. By the I-independent lift (Definition 6.4; Wang [16, Rem. 2.11], Bisi–Winkelmann [4, §3.7]) there is one holomorphic stem with $\mathbb{R}$-valued components,

$$A(x+Iy) = F_1(x+iy) + I\,F_2(x+iy), \qquad F_1, F_2 \ \mathbb{R}\text{-valued},$$

the same for every $I \in S^6$. Three consequences used below:

(i) *Slice preservation of values.* $A(\mathbb{C}_I^*) \subseteq \mathbb{C}_I^*$ for every I: the value at a point lies on that point's own slice sphere (Definition 5.1).

(ii) *G_2-equivariance.* For $g \in G_2$ and non-real $q = x + Iy$,

$$A(g \cdot q) = F_1 + g(I)\,F_2 = g \cdot \big(F_1 + I\,F_2\big) = g \cdot A(q),$$

since $g \cdot (x+Iy) = x + g(I)y$ (Definition 7.3) and F_1, F_2 are real; for $q \in \mathbb{R} \cup \{N\}$ both sides are G_2-fixed, because $A(\mathbb{R}) \subseteq \mathbb{R}$ and $A(N) \in \bigcap_I \mathbb{C}_I^* = \mathbb{R} \cup \{N\}$ by (i). (For $\zeta_{\mathbb{O}^*}$ this is Theorem 7.4; the computation above is the same one, run for an arbitrary intrinsic section.)

38. Definition - The section's slice data and equivariance (continued)

(iii) *Blindness to the sphere direction.* On a G_2-orbit $S_{(\sigma,\gamma)}$ the stem coordinates $(F_1, F_2)(\sigma + i\gamma)$ are constant; in particular if A vanishes at one point of the orbit it vanishes on the whole orbit (Lemma 10.2).

A slice-preserving section carries one holomorphic stem per slice, and the direction S^6 is traversed only by the G_2-morphisms already present in both worlds.

Lean declaration. ASection.realize
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize
'ASection.realize' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.realize_mem_sliceSphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize_mem_sliceSphere
'ASection.realize_mem_sliceSphere' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.realize_equivariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.realize_equivariant
'ASection.realize_equivariant' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.sliceCoord_smul_invariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

38. Definition - The section's slice data and equivariance (continued)

```
#print axioms ASection.sliceCoord_smul_invariant
'ASection.sliceCoord_smul_invariant' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

39. Definition - The octonionic and slice-value worlds

Author's statement

Recall the *translation groupoid* of an action $G \curvearrowright X$: its objects are the points of X and its morphisms $x \to y$ are the elements $g \in G$ satisfying $g \cdot x = y$. Thus $\pi_0(G \ltimes X) = X/G$ (Quillen [22, §1]; in Lean, `CategoryTheory.ActionCategory`).

The octonionic world is

$$\mathcal{H}_1 = G_2 \ltimes \mathbb{O}^*.$$

Its components are the G_2-orbits: the compactified real fixed locus $\mathbb{R} \cup \{N\}$ and, for each $\sigma \in \mathbb{R}$ and $\gamma > 0$, the residue-$\mathbb{C}$ sphere $S_{(\sigma,\gamma)} = \{\sigma + \gamma v : v \in S^6\}$, labelled by its real centre σ and radius γ (Baez [3]; Theorem 7.5).

The slice-value world $\mathcal{S}_2$ organizes the same compactified values by their slice decomposition. Every slice contains the real axis, and

$$\mathbb{O}^* = \bigcup_{I \in S^6} \mathbb{C}_I^*, \qquad \mathbb{C}_I^* \cap \mathbb{C}_J^* = \mathbb{R} \cup \{N\} \quad (J \neq \pm I).$$

Its objects are the points of $\mathbb{O}^*$. Its generating paths are the direction arrows induced by G_2 and the band phases $z \mapsto e^{I\theta} z$ within a compactified slice; the Lean object `S2` is the quotient path category by the direction-composition relations. In particular 0 and N are shared values, not new objects indexed by a zero.

These two geometric views hold the octonionic and slice-value geometry drawn on below.

Lean declaration. `H1`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms H1
'H1' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `SphereWorld`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

39. Definition - The octonionic and slice-value worlds (continued)

Lean command and literal axiom print.

```
#print axioms SphereWorld
'SphereWorld' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

40. Definition - The projective base, genuine section functor, and total category

Author's statement

The base is the projective action groupoid

$$\mathcal{B} = PGL(2,\mathbb{R}) \ltimes \mathrm{OnePoint}(\mathbb{R}).$$

Its objects are points of the compactified real projective line and its arrows are projective transformations together with their transport equations.

The matrix construction makes explicit why the analytic object is simultaneously a function and a group element. Let

$$C = \begin{pmatrix} 1 & -i \\ 1 & i \end{pmatrix}, \qquad M(z) = \begin{pmatrix} e^z & 0 \\ 0 & 1 \end{pmatrix}, \qquad D(z) = \left[C\, M(z)\, C^{-1}\right] \in \mathrm{Möb}(\mathbb{C}^*).$$

The matrix C is the Cayley map $w \mapsto (w-i)/(w+i)$. Matrix multiplication and $e^{z+w} = e^z e^w$ give

$$M(z+w) = M(z)M(w), \qquad D(z+w) = D(z)D(w).$$

Thus exponentiation is the homomorphism which turns addition of logarithmic lifts into composition of the actual slice Möbius maps. In the Lean source these are the chain `diagonalGL`, `diagonalGLHom`, `expUnit`, and `diskExpAction`, with their multiplication laws. Conditions C1–C3 select the A-specific logarithm λ_A of the continued nonzero pole factor and hence the one element

$$D_A = D(\lambda_A) = \left[C\,\mathrm{diag}(e^{\lambda_A}, 1)C^{-1}\right].$$

The Euler and Weierstrass expressions are the two presentations of this same multiplier. The function/group-element passage is the commuting dictionary

$$\begin{array}{ccc} z & \xrightarrow{\quad\exp\quad} & e^z \in \mathbb{C}^\times \\ \big\downarrow & & \big\downarrow \\ M(z) = \mathrm{diag}(e^z, 1) & \xrightarrow[C(-)C^{-1}]{} & D(z) \in \mathrm{Möb}(\mathbb{C}^*). \end{array}$$

The common Euler–Weierstrass multiplier can be followed directly through this dictionary. Write p_A for the pole of A and $g_A =$ `distinguishedPoleFactor A`. On the punctured pole

40. Definition - The projective base, genuine section functor, and total category (continued)

chart, conditions C2 and C3 give the two identities

$$g_A(z) = (z - p_A)\exp\left(\sum_{p\in A.\iota} \ell_p(z)\right),$$
$$g_A(z) = z^{m_A} R_{\mathrm{fac},A}(z) e^{h_A(z)} \prod_{n\geq 0} \mathcal{E}\big(z; q_{A,n}\big).$$

Condition C1 continues this same factor through the simple pole and proves $u_A = g_A(p_A) \neq 0$. Hence $u_A \in \mathbb{C}^\times$, its logarithm $\lambda_A = \log u_A$ satisfies $e^{\lambda_A} = u_A$, and the distinguished element is

$$D_A = D(\lambda_A) = \left[C \begin{pmatrix} u_A & 0 \\ 0 & 1 \end{pmatrix} C^{-1}\right].$$

This calculation also gives its two fixed boundary points. The diagonal matrix $\mathrm{diag}(u_A, 1)$ fixes 0 and ∞ in the standard projective chart; conjugation by C carries those points to the projective zero and the shared north point N. Therefore

$$D_A(0) = 0, \qquad D_A(N) = N,$$

which are the declarations

`ASection.distinguishedDiskAction_fixes_cayley_zero`,
`ASection.distinguishedDiskAction_fixes_cayley_N`.

The Euler expression at 0 and the Weierstrass expression at N are consequently the two boundary readings of this one matrix-built function.
The GPV lift records how that function varies between its boundary presentations. Let $X, Y \in \mathcal{B}$. A projective-base transport from X to Y carries three continuous tapes: a compactified domain path δ^*, a nonzero value path v, its compactification v^*, and a logarithmic lift Γ. Their defining equations are

$$\begin{aligned} \delta^*(0) &= \mathrm{cayleyCoord}(X), & \delta^*(1) &= \mathrm{cayleyCoord}(Y), \\ v^*(t) &= A^*\big(\delta^*(t)\big), & e^{\Gamma(t)} &= v(t), \\ \Gamma(1) - \Gamma(0) &= 2\pi i k, & k &\in \mathbb{Z}. \end{aligned}$$

These are precisely the fields of `ASection.GpvTransport`. At each t the lift equation passes through the matrix dictionary:

$$\begin{array}{ccc} \Gamma(t) & \xrightarrow{\quad\exp\quad} & v(t) \in \mathbb{C}^\times \\ {\scriptstyle D}\big\downarrow & & \big\downarrow{\scriptstyle u\mapsto[C\,\mathrm{diag}(u,1)C^{-1}]} \\ D(\Gamma(t)) & = & \big[C\,\mathrm{diag}(v(t),1)C^{-1}\big]. \end{array}$$

Thus every point of the lift is already an action of the same kind as D_A. Taking real parts and norms in the lift equation gives

$$\mathrm{Re}\,\Gamma(t) = \log\|v(t)\|.$$

40. Definition - The projective base, genuine section functor, and total category (continued)

The right-hand side shows that the real level is continuous and depends only on the value tape. Any second continuous lift with the same initial rung equals Γ, while every lift of the same value tape has this same real part. Taking real parts in the winding equation gives

$$\operatorname{Re}\Gamma(0) = \operatorname{Re}\Gamma(1),$$

and exponentiating the full equation gives

$$D(\Gamma(0)) = D(\Gamma(1)).$$

The winding integer therefore records the vertical displacement of the logarithmic rung, and the real level and the endpoint Möbius action remain fixed.
Condition C2 now supplies the canonical arithmetic instance of this construction. Along an Euler half-space loop δ,

$$\Gamma_A(t) = \sum_{p \in A.\iota} \ell_p(\delta(t)), \qquad e^{\Gamma_A(t)} = A(\delta(t)),$$

with the prime index retained inside the complete summation. Local normal convergence makes Γ_A continuous, the zero-free half-space makes its value tape lie in $\mathbb{C}^\times$, and the GPV uniqueness theorem fixes the whole lift once $\Gamma_A(0)$ is chosen. Since $\delta(0) = \delta(1)$, the same prime sum occurs at both endpoints, so its checked winding is $k = 0$. At every instant the square `canonicalAsectionPresentation_euler_toNorth` carries this complete prime tape to the common north frame. Condition C3 supplies the Weierstrass boundary presentation there, and condition C1 identifies both presentations with the continued factor g_A. The prime sum, its value under A, its logarithmic level and winding, and the matrix action it generates are therefore simultaneous coordinates of the distinguished element.
The projective action is read in the direction-indexed sphere-world groupoid. Its objects are directions $I \in S^6$ equipped with their compactified slice spheres, and its morphisms carry the direction and Möbius legs. A normalized value state pairs such a direction with a point of its slice. To position D_A over the whole base, fix the canonical orbit representative $o_b : N \to b$. Every arrow $f : x \to y$ has the unique residual north-stabilizer factor

$$r_f = o_y^{-1} f\, o_x, \qquad f = o_y r_f o_x^{-1}.$$

The object frame and the full arrow element are therefore

$$a_A(b) = C(o_b) D_A, \qquad a_A(f) = a_A(y)\, C(r_f)\, a_A(x)^{-1}.$$

Here $C(-)$ denotes the multiplicative Cayley-projective homomorphism. The fixed orbit representatives force r_f uniquely; $r_{1_x} = 1$ and $r_{f \gg g} = r_g r_f$. These identities, together with multiplicativity of $C(-)$, prove the identity and composition laws directly from the fixed representatives. This is the orbit–stabilizer well-definedness of both functors below.
The preceding construction gives nine checkable readings on the projective base:

(B1) the C2 Euler and C3 Weierstrass formulas present the one C1-continued factor g_A;

(B2) $u_A = g_A(p_A)$ and $\lambda_A = \log u_A$ generate the one matrix action D_A;

40. Definition - The projective base, genuine section functor, and total category (continued)

(B3) the diagonal calculation fixes the two boundary points 0 and N;

(B4) the Euler lift retains the complete prime-indexed sum $\sum_{p\in A.\iota} \ell_p$;

(B5) exponentiation identifies the lift pointwise with the A-value and with its Möbius matrix;

(B6) the initial rung determines the unique tame continuous lift;

(B7) its real level is $\log\|A\|$, continuous and independent of the lift branch;

(B8) its winding lies in $2\pi i\mathbb{Z}$, preserves the endpoint action and real level, and equals zero on the C2 Euler loop;

(B9) the unique orbit and stabilizer factors carry every instant of this action through the base, with identity and composition inherited from their factorization laws.

Each line is a consequence of the construction above: (B1)–(B3) come from the common pole factor and its matrix, (B4)–(B8) from its GPV lift, and (B9) from the projective orbit–stabilizer sweep.
The direction-and-Möbius projection is

$$A^{\mathrm{slice}} : \mathcal{B} \longrightarrow \mathrm{SphereWorld},$$

written `AsectionSlice A` in the Lean source. It records the slice sphere and its Möbius transport. The full A-section functor below additionally retains the normalized point and the value produced on that sphere.
The slice-preserving realization on $\mathbb{O}^*$ is the G_2-equivariant endofunctor $A_{\mathbb{O}} : \mathcal{H}_1 \to \mathcal{H}_1$. The normalized state world has functorial input and output eyes, and the round trip is the checked triangle

$$\begin{array}{ccc} \mathrm{StateWorld}_A & \xrightarrow{\mathrm{In}_A} & \mathcal{H}_1 \\ & \searrow^{\mathrm{Out}_A} & \downarrow{\scriptstyle A_{\mathbb{O}}} \\ & & \mathcal{H}_1, \end{array} \qquad \mathrm{Out}_A = \mathrm{In}_A \gg A_{\mathbb{O}}.$$

This equality is `AsectionState_input_then_equivariant`: it holds on arrows as well as objects, so the normalized input coordinates genuinely drive the realized A-values.
Positioning this round trip over $b \in \mathcal{B}$ forms a constrained graph. An object in the fibre consists of an input state x, the forced positioned state $a_A(b)\cdot x$, and the forced value $\mathrm{Out}_A(a_A(b)\cdot x)$. Thus both the positioned point and its value are determined by the input and the action. For $f : b \to b'$, the two legs of the orbit–stabilizer square transport the input and positioned faces, and its already-proved commuting equation makes the target graph well-defined. Reading that same square through In_A and Out_A gives the checked input and output naturality squares. Identity and composition of these graph transports are inherited from the unique factorization of r_f. Sweeping the whole constrained graph through the base therefore gives the genuine A-section functor

$$\mathcal{A}_A : \mathcal{B} \longrightarrow \mathbf{Grpd},$$

written `AsectionActionDiagram A` in the Lean source.

40. Definition - The projective base, genuine section functor, and total category (continued)

The nine base readings (B1)–(B9) pass through this constrained graph by the certified projective/GPV action squares. Three further readings are thereby realized in the generated states. First, condition C3 supplies the populated residue-$\mathbb{C}$ sphere states, represented by `residueActionState` and certified by `residueActionState_mem`. Second, condition C4 makes their population infinite through `c4_infinite`. Third, every such state carries its intrinsic real coordinate through `transportLevel`. These are the three output readings. Together with (B1)–(B9) they form the $9 + 3$ partition: nine properties of the one distinguished action on the base, followed functorially by three readings of the A-generated residue states in the total. The total value-transport category is

$$\mathcal{T}_A = \int_{\mathcal{B}} \mathcal{A}_A.$$

An object is a base position together with a normalized A-section value state in its fibre. A morphism is a base channel together with the compatible A-section transport supplied by A. The populated residue-$\mathbb{C}$ zero-sphere states are the degenerate-fibre outputs of this completed construction, produced by C3–C4. The associated diagram $\pi_0 \circ \mathcal{A}_A : \mathcal{B} \to \mathbf{Set}$ is the components diagram required by the Grothendieck/colimit readout (Lemma 10.3).

Lean declaration. `GreatCircle.Base`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.Base
'GreatCircle.Base' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `GreatCircle.cayleyGL`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.cayleyGL
'GreatCircle.cayleyGL' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `GreatCircle.diagonalGL`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms GreatCircle.diagonalGL
'GreatCircle.diagonalGL' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. GreatCircle.diagonalGL_mul
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.diagonalGL_mul
'GreatCircle.diagonalGL_mul' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.diagonalGLHom
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.diagonalGLHom
'GreatCircle.diagonalGLHom' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.expUnit
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.expUnit
'GreatCircle.expUnit' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. GreatCircle.expUnit_add
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.expUnit_add
'GreatCircle.expUnit_add' depends on axioms: [propext, Classical.choice, Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. GreatCircle.diskExpAction
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.diskExpAction
'GreatCircle.diskExpAction' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.diskExpAction_add
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.diskExpAction_add
'GreatCircle.diskExpAction_add' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.cayleyProjective
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.cayleyProjective
'GreatCircle.cayleyProjective' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.cayleyProjective_mul
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.cayleyProjective_mul
'GreatCircle.cayleyProjective_mul' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. GreatCircle.orbitRep
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.orbitRep
'GreatCircle.orbitRep' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. GreatCircle.orbitRep_spec
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.orbitRep_spec
'GreatCircle.orbitRep_spec' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.stabilizerPart
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.stabilizerPart
'GreatCircle.stabilizerPart' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.orbit_stabilizer_factor
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.orbit_stabilizer_factor
'GreatCircle.orbit_stabilizer_factor' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.stabilizerPart_unique
Author's object.

40. Definition - The projective base, genuine section functor, and total category (continued)

Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.stabilizerPart_unique
'GreatCircle.stabilizerPart_unique' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.stabilizerPart_id
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.stabilizerPart_id
'GreatCircle.stabilizerPart_id' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. GreatCircle.stabilizerPart_comp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms GreatCircle.stabilizerPart_comp
'GreatCircle.stabilizerPart_comp' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.distinguishedDiskAction
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.distinguishedDiskAction
'ASection.distinguishedDiskAction' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.distinguishedDiskAction_eq_fullMultiplier
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.distinguishedDiskAction_eq_fullMultiplier
'ASection.distinguishedDiskAction_eq_fullMultiplier' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.distinguishedPoleFactor_euler
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.distinguishedPoleFactor_euler
'ASection.distinguishedPoleFactor_euler' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.distinguishedPoleFactor_weierstrass
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.distinguishedPoleFactor_weierstrass
'ASection.distinguishedPoleFactor_weierstrass' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.eulerPrimeSum
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.eulerPrimeSum
'ASection.eulerPrimeSum' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.eulerDiskAction_eq_value
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.eulerDiskAction_eq_value
'ASection.eulerDiskAction_eq_value' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.GpvTransport
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport
'ASection.GpvTransport' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.diskExpAction_eq_value
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.diskExpAction_eq_value
'ASection.GpvTransport.diskExpAction_eq_value' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.diskExpAction_endpoint_eq
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.diskExpAction_endpoint_eq
'ASection.GpvTransport.diskExpAction_endpoint_eq' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.lift_endpoint_re_eq
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.lift_endpoint_re_eq
'ASection.GpvTransport.lift_endpoint_re_eq' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.level
Author's object.

40. Definition - The projective base, genuine section functor, and total category (continued)

Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.level
'ASection.GpvTransport.level' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.GpvTransport.continuous_level
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.continuous_level
'ASection.GpvTransport.continuous_level' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.lift_unique
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.lift_unique
'ASection.GpvTransport.lift_unique' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.GpvTransport.level_independent
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.level_independent
'ASection.GpvTransport.level_independent' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.value_at_source
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.GpvTransport.value_at_source
'ASection.GpvTransport.value_at_source' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.value_at_target
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.value_at_target
'ASection.GpvTransport.value_at_target' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.endpoint_log_norm_eq
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.endpoint_log_norm_eq
'ASection.GpvTransport.endpoint_log_norm_eq' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.GpvTransport.endpoint_norm_eq
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.GpvTransport.endpoint_norm_eq
'ASection.GpvTransport.endpoint_norm_eq' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.projectiveGpvActionSquare
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.projectiveGpvActionSquare
'ASection.projectiveGpvActionSquare' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.projectiveGpvActionSquare_level
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveGpvActionSquare_level
'ASection.projectiveGpvActionSquare_level' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.projectiveGpvActionSquare_input_commutes
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveGpvActionSquare_input_commutes
'ASection.projectiveGpvActionSquare_input_commutes' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.projectiveGpvActionSquare_output_commutes
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveGpvActionSquare_output_commutes
'ASection.projectiveGpvActionSquare_output_commutes' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_prime_stack
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.canonicalAsectionPresentation_euler_prime_stack
'ASection.canonicalAsectionPresentation_euler_prime_stack' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_lift_exp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.canonicalAsectionPresentation_euler_lift_exp
'ASection.canonicalAsectionPresentation_euler_lift_exp' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_lift_unique
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.canonicalAsectionPresentation_euler_lift_unique
'ASection.canonicalAsectionPresentation_euler_lift_unique' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_level_continuous
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.canonicalAsectionPresentation_euler_level_continuous
'ASection.canonicalAsectionPresentation_euler_level_continuous' depends on axioms:
↪  [propext,
 Classical.choice,
 Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_winding
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.canonicalAsectionPresentation_euler_winding
'ASection.canonicalAsectionPresentation_euler_winding' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.canonicalAsectionPresentation_euler_toNorth
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.canonicalAsectionPresentation_euler_toNorth
'ASection.canonicalAsectionPresentation_euler_toNorth' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.projectiveObjectFrame
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveObjectFrame
'ASection.projectiveObjectFrame' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.projectiveArrowElement
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveArrowElement
'ASection.projectiveArrowElement' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.projectiveObjectFrame_north
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.projectiveObjectFrame_north
'ASection.projectiveObjectFrame_north' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.distinguishedDiskAction_fixes_cayley_zero
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.distinguishedDiskAction_fixes_cayley_zero
'ASection.distinguishedDiskAction_fixes_cayley_zero' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.distinguishedDiskAction_fixes_cayley_N
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.distinguishedDiskAction_fixes_cayley_N
'ASection.distinguishedDiskAction_fixes_cayley_N' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.projectiveArrowElement_id
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.projectiveArrowElement_id
'ASection.projectiveArrowElement_id' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.projectiveArrowElement_comp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

40. Definition - The projective base, genuine section functor, and total category (continued)

```
#print axioms ASection.projectiveArrowElement_comp
'ASection.projectiveArrowElement_comp' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionSlice
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionSlice
'ASection.AsectionSlice' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionEquivariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionEquivariant
'ASection.AsectionEquivariant' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionStateInput
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionStateInput
'ASection.AsectionStateInput' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionStateOutput
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionStateOutput
'ASection.AsectionStateOutput' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.AsectionState_input_then_equivariant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionState_input_then_equivariant
'ASection.AsectionState_input_then_equivariant' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.ActionTransportSquare.coordinateTransport_commutes
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.ActionTransportSquare.coordinateTransport_commutes
'ASection.ActionTransportSquare.coordinateTransport_commutes' depends on axioms:
↪  [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.ActionTransportSquare.input_commutes
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.ActionTransportSquare.input_commutes
'ASection.ActionTransportSquare.input_commutes' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.ActionTransportSquare.output_commutes
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.ActionTransportSquare.output_commutes
'ASection.ActionTransportSquare.output_commutes' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.AsectionActionState
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionState
'ASection.AsectionActionState' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionActionTransport
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport
'ASection.AsectionActionTransport' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionActionState.ofInput
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionState.ofInput
'ASection.AsectionActionState.ofInput' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionActionState_ofInput_value
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionState_ofInput_value
'ASection.AsectionActionState_ofInput_value' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.AsectionActionTransport_obj_input
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport_obj_input
'ASection.AsectionActionTransport_obj_input' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionActionTransport_obj_positioned
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport_obj_positioned
'ASection.AsectionActionTransport_obj_positioned' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionActionTransport_obj_value
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport_obj_value
'ASection.AsectionActionTransport_obj_value' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionActionTransport_id
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport_id
'ASection.AsectionActionTransport_id' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.AsectionActionTransport_comp
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionTransport_comp
'ASection.AsectionActionTransport_comp' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionActionDiagram
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionActionDiagram
'ASection.AsectionActionDiagram' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.residueActionState
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.residueActionState
'ASection.residueActionState' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.residueActionState_mem
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.residueActionState_mem
'ASection.residueActionState_mem' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

40. Definition - The projective base, genuine section functor, and total category (continued)

Lean declaration. ASection.c4_infinite
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.c4_infinite
'ASection.c4_infinite' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.transportLevel
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.transportLevel
'ASection.transportLevel' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.ambientTotalCategory
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.ambientTotalCategory
'ASection.ambientTotalCategory' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.ambientTotalGroupoid
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.ambientTotalGroupoid
'ASection.ambientTotalGroupoid' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. grothendieckGrpdGroupoid
Author's object.

40. Definition - The projective base, genuine section functor, and total category (continued)

Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms grothendieckGrpdGroupoid
'grothendieckGrpdGroupoid' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. CategoryTheory.Grothendieck
Author's object.
Object relation. Object relation not checked
Exact state. External library dependency; approved kernel-reported axiom surface; no authored-object anchor required

Lean command and literal axiom print.

```
#print axioms CategoryTheory.Grothendieck
'CategoryTheory.Grothendieck' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

41. Proposition - Slice-regular Weierstrass factorization; the content of C3

Author's statement
Let $A \in \mathcal{R}$ have a zero of order $m \geq 0$ at the origin and no non-real isolated zeros (for $A \in \mathcal{R}$ the latter is automatic: non-real zeros come in whole spheres, Lemma 10.2). Then on $\mathbb{O}^* \setminus \{N\}$ it factors as

$$A = q^m \, R \, S \, h,$$

where q denotes the octonionic coordinate function (the identity section, so q^m carries the order-m zero at the origin), R is a slice-preserving factor vanishing exactly at the real zeros of A, S is a factor vanishing exactly at its spherical (residue-$\mathbb{C}$) zero-spheres, and h is never vanishing; each of R, S is the slice-regular Weierstrass product over the corresponding zeros. The factorization holds *including the case where either family of zeros is infinite.*

Lean declaration. weierstrassE
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms weierstrassE
```

41. Proposition - Slice-regular Weierstrass factorization; the content of C3 (continued)

```
'weierstrassE' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. spherePrimary
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms spherePrimary
'spherePrimary' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.c3_multipliable
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.c3_multipliable
'ASection.c3_multipliable' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.c3_locMajorant
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.c3_locMajorant
'ASection.c3_locMajorant' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. ASection.c3_factorization
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.c3_factorization
'ASection.c3_factorization' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

42. Lemma - Residue-$\CC$ zeros are 6-sphere components of $\mathcal H_1$

Author's statement
For $A \in \mathcal{R}$, the non-real zeros of A are 6-spheres, each a single connected component of the translation groupoid $\mathcal{H}_1$.

Lean declaration. ASection.sphereZero_mem_CResidueZeroLocus
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.sphereZero_mem_CResidueZeroLocus
'ASection.sphereZero_mem_CResidueZeroLocus' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.mem_CResidueZeroLocus_iff_exists_sphereZero
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.mem_CResidueZeroLocus_iff_exists_sphereZero
'ASection.mem_CResidueZeroLocus_iff_exists_sphereZero' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. G2.exists_smul_eq_of_mem_unitImaginarySphere
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms G2.exists_smul_eq_of_mem_unitImaginarySphere
'G2.exists_smul_eq_of_mem_unitImaginarySphere' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

43. Lemma - π_0 of a Grothendieck construction

Author's statement
For a functor $F : \mathcal{B} \to \mathrm{Grpd}$, the connected-components functor carries the Grothendieck

43. Lemma - π_0 of a Grothendieck construction (continued)

construction to the colimit of the component diagram:

$$\pi_0\Big(\int_{\mathcal{B}} F\Big) \;\cong\; \operatorname*{colim}_{\mathcal{B}} (\pi_0 \circ F).$$

Lean declaration. CategoryTheory.Grothendieck
Author's object.
Object relation. Object relation not checked
Exact state. External library dependency; approved kernel-reported axiom surface; no authored-object anchor required

Lean command and literal axiom print.

```
#print axioms CategoryTheory.Grothendieck
'CategoryTheory.Grothendieck' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. pi0GrothendieckEquiv
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms pi0GrothendieckEquiv
'pi0GrothendieckEquiv' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. pi0_grothendieck
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms pi0_grothendieck
'pi0_grothendieck' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

44. Definition - A-sections

Author's statement
An *A-section* is a section A of the ring $\mathcal{R}$ of slice-preserving slice-regular functions on $\mathbb{O}^* = S^8$ (Definition 5.1) with the following four properties.

(C1) *Meromorphic continuation; single simple pole.* A is meromorphically continued to $\mathbb{O}^*$

44. Definition - A-sections (continued)

with exactly one pole; it is simple, lies at a real point, and its value there is the point at infinity $\infty = N$.

(C2) *Infinite Euler product.* On a slice right half-space $\Omega_0 \subset \mathbb{O}^*$, $A = \exp\big(\sum_p \ell_p\big)$ for an *infinite* family $\{\ell_p\}$ of slice-preserving slice-regular functions each zero-free on Ω_0, the family summable *locally normally* on Ω_0 (the sense in which the cited Euler products converge; Theorem 1.2); in particular A is zero-free on Ω_0.

(C3) *Infinite Weierstrass factorization.* Over the *full divisor* of A, including the single pole of (C1), at the real point p_0, multiplicity -1, carried by the explicit left factor, on $\mathbb{O}^* \setminus \{p_0\}$,

$$(q - p_0)\, A \;=\; q^m\, R\, e^g \prod_n \mathcal{E}(\,\cdot\,; q_n),$$

where q is the octonionic coordinate, $m \geq 0$, R is the slice-regular Weierstrass product over the *nonzero* residue-$\mathbb{R}$ (real) zeros of A, vanishing only on $\mathbb{R}$, g is slice-preserving entire, $\{q_n\}$ enumerates the residue-$\mathbb{C}$ zero-spheres of A, and $\mathcal{E}(\,\cdot\,; q_n)$ is the slice-regular Weierstrass primary factor of q_n (Proposition 10.1; [2, Prop. 3.1, Thm. 3.2], [10]); the factorization holds with either family infinite, the product converging *locally normally* on $\mathbb{O}^* \setminus \{p_0\}$ (the convergence of Proposition 10.1). (The left factor makes "full divisor" literal: the divisor of a meromorphic section includes its single pole; classically, Hadamard factors $(s-1)\zeta(s)$.)

(C4) *Infinitely many residue-$\mathbb{C}$ zeros.* The index set $\{q_n\}$ is infinite. A closed zero of such a section is a real point (residue field $\mathbb{R}$) or a 6-sphere $S_{(\sigma,\gamma)}$ (residue field $\mathbb{C}$; [2]); residue-$\mathbb{C}$ names the classically nontrivial zeros (Part 2).

Lean declaration. `ASection`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection
'ASection' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean declaration. `weierstrassE`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms weierstrassE
'weierstrassE' depends on axioms: [propext, Classical.choice, Quot.sound]
```

---

44. Definition - A-sections (continued)

Lean declaration. spherePrimary
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms spherePrimary
'spherePrimary' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

45. Definition - The residue subdiagram and its inclusion

Author's statement
The *residue locus* of an A-section is $\{z : A(z) = 0,\ \operatorname{Im} z > 0\}$ (`CResidueZeroLocus`). The *residue subdiagram* $\mathcal{R}_A : \mathcal{B} \to \mathbf{Grpd}$ (`AsectionCResidueDiagram`) is the preimage of the states this locus selects at the north frame — a preimage taken in the total action groupoid, not of a set — with the witnessing base arrow carried as data; its inclusion $\iota_A : \mathcal{R}_A \Rightarrow \mathcal{A}_A$ (`AsectionCResidueInclusion`) is the fibrewise fully faithful inclusion onto that inverse image. Its naturality squares retain the ambient A-section transport itself.

Lean declaration. ASection.CResidueZeroLocus
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.CResidueZeroLocus
'ASection.CResidueZeroLocus' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionCResidueDiagram
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionCResidueDiagram
'ASection.AsectionCResidueDiagram' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionCResidueInclusion

45. Definition - The residue subdiagram and its inclusion (continued)

Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionCResidueInclusion
'ASection.AsectionCResidueInclusion' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.AsectionCResidueInclusionTotal
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionCResidueInclusionTotal
'ASection.AsectionCResidueInclusionTotal' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionCResidueInclusionTotal_full
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionCResidueInclusionTotal_full
'ASection.AsectionCResidueInclusionTotal_full' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.AsectionCResidueInclusionTotal_faithful
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.AsectionCResidueInclusionTotal_faithful
'ASection.AsectionCResidueInclusionTotal_faithful' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.residueTotalCategory
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

45. Definition - The residue subdiagram and its inclusion (continued)

```
#print axioms ASection.residueTotalCategory
'ASection.residueTotalCategory' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.residueTotalGroupoid
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.residueTotalGroupoid
'ASection.residueTotalGroupoid' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

46. Lemma - Transitivity of the C-residue total

Author's statement
For an A-section A, the total inverse-image groupoid

$$\int_{\mathcal{B}} \mathcal{R}_A$$

is transitive: any two of its objects are joined by a morphism.

Lean declaration. ASection.sweepTransitive_on_residueSystem
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.sweepTransitive_on_residueSystem
'ASection.sweepTransitive_on_residueSystem' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.northRelativeLoop_maps
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.northRelativeLoop_maps
```

46. Lemma - Transitivity of the C-residue total (continued)

```
'ASection.northRelativeLoop_maps' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.northComparison_of_parallelFaces
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.northComparison_of_parallelFaces
'ASection.northComparison_of_parallelFaces' depends on axioms: [propext,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

47. Theorem - Concentricity

Author's statement
Let A be an A-section. The infinitely many C-residue zero S^6 spheres transported by the A-section functor $\mathcal{A}_A : \mathcal{B} \to \mathbf{Grpd}$, from the projective great-circle base $\mathcal{B}$ to the A-generated normalized value groupoids, form exactly one colimit/component class, and that singleton class carries one real value

$$c \in \mathbb{R}.$$

Consequently every populated residue-$\mathbb{C}$ zero 6-sphere has centre c, so the infinitely many populated residue-$\mathbb{C}$ zero spheres are concentric.

Lean declaration. ASection.concentricity
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.concentricity
'ASection.concentricity' depends on axioms: [propext, sorryAx, Classical.choice,
↪  Quot.sound]
```

Lean declaration. ASection.residueTotal_isConnected
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.residueTotal_isConnected
```

47. Theorem - Concentricity (continued)

```
'ASection.residueTotal_isConnected' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.residueTotal_pi0_singleton
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.residueTotal_pi0_singleton
'ASection.residueTotal_pi0_singleton' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Lean declaration. ASection.transportLevel
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.transportLevel
'ASection.transportLevel' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

48. Corollary - Translation to the classical framework

Author's statement
Under the residue-$\mathbb{C}$/classically-nontrivial-zero identification, the already-proved populated residue-$\mathbb{C}$ zero spheres are the classically nontrivial zero spheres and retain the common centre c supplied by Theorem 10.7.

Lean declaration. ASection.nontrivial_one_centre
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms ASection.nontrivial_one_centre
'ASection.nontrivial_one_centre' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

49. Corollary - ζO is an A-section

Author's statement
$\zeta_{\mathbb{O}^*}$ is an A-section (Definition 10.4): a section of $\mathcal{R}$ satisfying (C1)–(C4).

Lean declaration. `zetaSection`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted; approved axiom surface; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zetaSection
'zetaSection' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master

50. Corollary - Riemann Hypothesis

Author's statement
Every nontrivial zero of the classical Riemann zeta function has real part $\frac{1}{2}$; in particular infinitely many zeros lie on the line $\Re s = \frac{1}{2}$.

Lean declaration. `zeta_riemannHypothesis`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeta_riemannHypothesis
'zeta_riemannHypothesis' depends on axioms: [propext, sorryAx, Classical.choice,
↪  Quot.sound]
```

Lean declaration. `zeta_criticalLine_zeros_infinite`
Author's object.
Object relation. Object relation not checked
Exact state. Lean declaration accepted with sorryAx; authored-object anchor unratified

Lean command and literal axiom print.

```
#print axioms zeta_criticalLine_zeros_infinite
'zeta_criticalLine_zeros_infinite' depends on axioms: [propext, sorryAx,
↪  Classical.choice, Quot.sound]
```

Receipt import. AUTHOR_BINDING_REGISTRY_REJECTED: the ratified author binding registry does not match the current master